Linux File Permissions and Access Control

1. File Permissions

Linux provides a **file permission mechanism** to control who can access a file or directory and what operations they can perform.

The three basic permissions are:

Permission	Symbol	Meaning
Read	`r`	Read or view the contents
Write	`w`	Modify the contents
Execute	`x`	Execute the file/program

User Categories

Linux permissions are assigned to three categories of users:

Category	Symbol	Meaning
User	`u`	Owner of the file
Group	`g`	Group associated with the file
Others	`o`	All other users

---

2. Understanding File Permissions

- Consider the following output:

```
-rwxr-xr--
```

- The permission string can be divided as:

```
-    rwx    r-x    r--
     |      |      |
     |      |      └── Others
     |      └───────── Group
     └──────────────── Owner
```

Explanation

- **Owner** → `rwx` → Read + Write + Execute
- **Group** → `r-x` → Read + Execute

- **Others** → `r--` → Read only

---

3. `chmod` Command

`chmod` stands for **Change Mode**.

- It is used to change the permissions of a file or directory.

Syntax

```
chmod [permissions] filename
```

Example

```
chmod u+x program.sh
```

This gives execute permission to the owner.

Before

```
-rw-r--r--
```

After

```
-rwxr--r--
```

---

4. Symbolic Mode with `chmod`

- Permissions can be modified using:

Symbol	Meaning
u	User / Owner
g	Group
o	Others
a	All users

`

`

Operators

Operator	Meaning
`+`	Add permission
`-`	Remove permission
`=`	Set exact permission

Examples

Give execute permission to the owner

```
chmod u+x program.sh
```

Remove write permission from the group

```
chmod g-w file.txt
```

Give read permission to others

```
chmod o+r file.txt
```

Give execute permission to everyone

```
chmod a+x program.sh
```

Set read and write permission for the owner

```
chmod u=rw file.txt
```

`

`

`

`

5. Numeric Mode with `chmod`

- Linux also represents permissions using numbers.

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1

The values are added together.

Permission Values

```
r-- = 4
-w- = 2
--x = 1

rw- = 4 + 2 = 6
r-x = 4 + 1 = 5
-wx = 2 + 1 = 3
rwx = 4 + 2 + 1 = 7
```

Example

```
chmod 0755 program.sh
```

Here:

```
7 → Owner
5 → Group
5 → Others
```

- Therefore:

```
7 = rwx
5 = r-x
4 = r--
```

So:

```
0755 → rwxr-xr-x
```

Meaning

- **Owner** → Read + Write + Execute
- **Group** → Read + Execute
- **Others** → Read + Execute

---

6. chown Command

`chown` stands for **Change Owner**.

- It is used to change the ownership of a file or directory.

Syntax

```
chown username filename
```

Example

```
chown user1 file.txt
```

This changes the owner of `file.txt` to `user1`.

---

7. Changing Owner and Group

We can change both the owner and group using:

Syntax

```
chown username:groupname filename
```

Example

```
chown user1:grp1 file.txt
```

- This means:

```
Owner → user1
Group → grp1
```

---

8. Checking File Ownership and Permissions

The `ls -l` command can be used to view file permissions and ownership.

```
ls -l
```

Example Output

```
    -rwxr-xr-- 1 user1 grp1 1024 Oct 1 10:30 program.sh
    Here:
        -           → File type
        rwxr-xr--   → Permissions
        1           → Number of links
        1024        → File size in bytes
        Oct 1 10:30 → Last modified date and time
        user1       → Owner
        grp1        → Group
        program.sh  → File name
```

---

9. Access Control List (ACL)

ACL stands for **Access Control List**.

- Normal Linux permissions provide access control for:
- Owner
- Group
- Others

However, sometimes we need to give special permissions to individual users or groups.

Example

```
file.txt

Owner  → Read + Write
Group  → Read
Others → No access
```

- Suppose we also want:

```
grp1 → Read + Write
grp2 → Read only
```

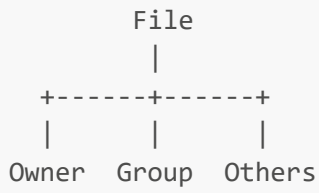
- Normal Linux permissions cannot directly provide separate permissions to grp1 and grp2.

For this purpose, Linux provides **ACL (Access Control List)**.

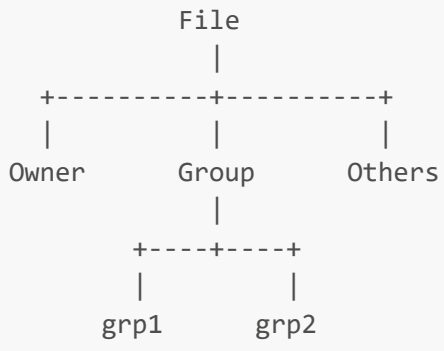
---

10. ACL Structure

Normal Permissions



ACL Provides More Detailed Control



- ACL allows permissions to be assigned to:
- Specific users
- Specific groups

---

11. `getfacl` Command

The `getfacl` command is used to view ACL information.

Syntax

```
getfacl filename
```

Example

```
getfacl file.txt
```

It displays the access control information associated with the file.

---

12. `setfacl` Command

The `setfacl` command is used to set or modify ACL permissions.

Syntax

```
setfacl -m u:username:permissions filename
```

Example

```
setfacl -m u:user1:rw file.txt
```

This gives:

```
User1 → Read + Write
```

permission on `file.txt`.

Another Example

```
setfacl -m u:user2:r file.txt
```

- This gives:

```
User2 → Read only
```

- permission on `file.txt`.
